

DISEÑO Y PRUEBAS

DESARROLLO ORIENTADO POR OBJETOS

Juan Diego Carreño Gutiérrez

Juan Diego Castaño Parra

ESCUELA COLOMBIANA DE INGENIERÍA

2026-2 — Laboratorio 2/6

BOGOTÁ D.C

ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS

Diseño y Pruebas.
2026-2 — Laboratorio 2/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Desarrollar una aplicación aplicando Desarrollo Orientado al Comportamiento (BDD) y Desarrollo Dirigido por Modelos (MDD).
2. Realizar diseños (ingeniería directa e inversa) utilizando una herramienta de modelado (astah)
3. Manejar pruebas de unidad usando un framework ([junit](#))
4. Apropiar nuevas clases consultando sus especificaciones ([API java](#))
5. Experimentar las prácticas XP :
Designing. Use [CRC](#) cards for design sessions.
Testing. All code must have [unit tests](#).

ENTREGA

- Incluyan en un archivo **.zip** los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (CasallasC-DuarteJ)
- Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Objetivo: Desarrollar una versión simplificada de [Tunes: miniTunes](#)

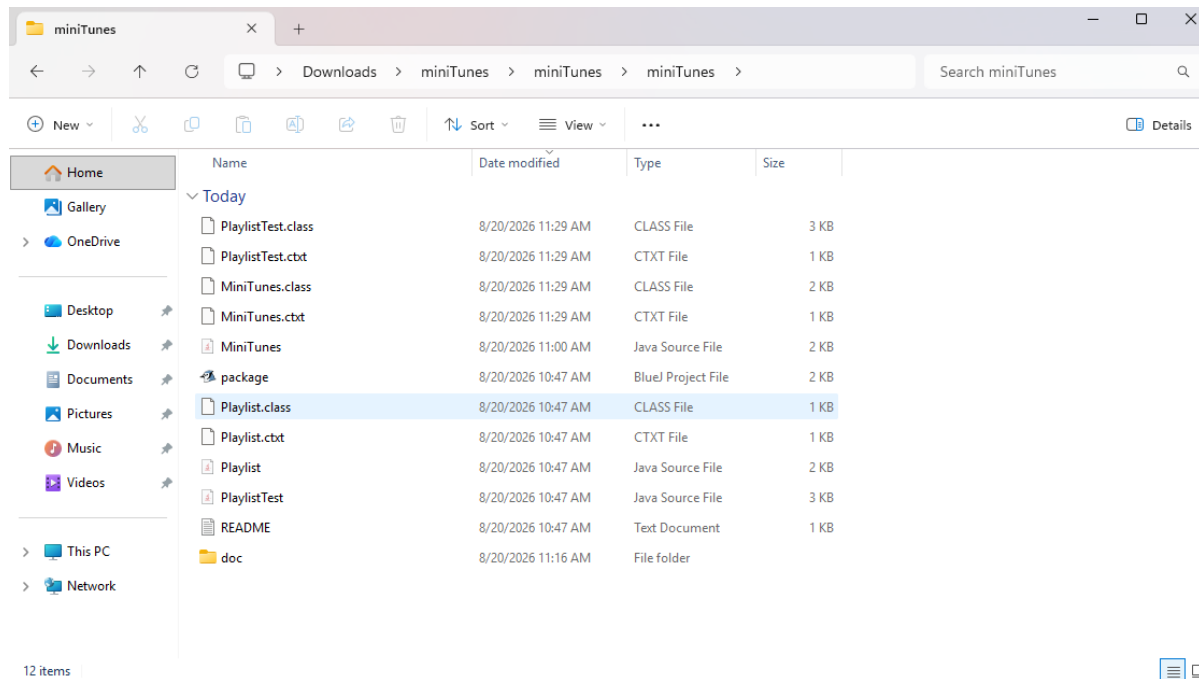
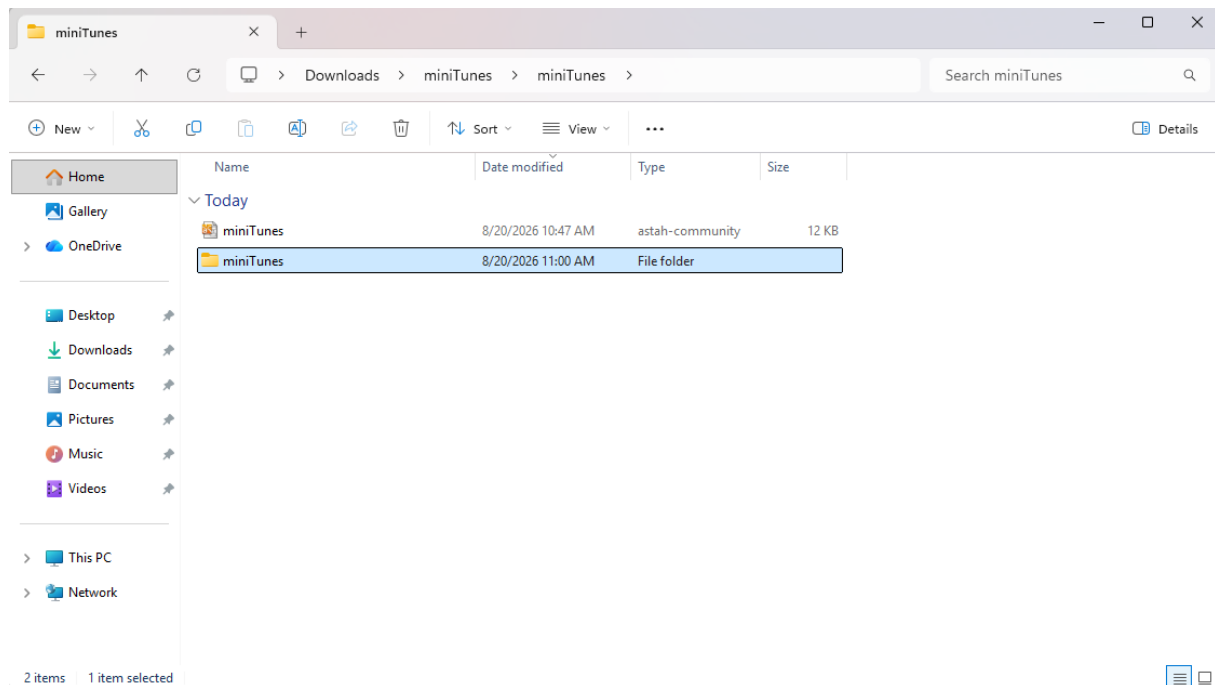
[iTunes](#) es una aplicación para administrar colecciones musicales basada en estructuras como canciones, álbumes y listas de reproducción, que permiten organizar y manipular información musical de forma eficiente.
En este laboratorio desarrollaremos una versión simplificada de [iTunes](#) utilizando únicamente listas de reproducción
Una lista de reproducción es una colección ordenada de canciones sobre la cual pueden realizarse operaciones de consulta, selección y combinación.
En [miniTunes](#) cada canción estará caracterizada por su título, artista, género, duración y calificación. [iTunes/Music Tutorial](#)



PARTE I. Conociendo el proyecto

A Revisando línea base [En [lab02.pdf](#)]

1. El proyecto "[miniTunes](#)" contiene una construcción parcial del sistema. Revisen el directorio donde se encuentra el proyecto. Describan el contenido en términos de (a) directorios y de (b) las extensiones de los archivos.
- A) En términos de directorio, en el proyecto se encuentra uno que contiene los diagramas necesarios del proyecto y un directorio que contiene lo necesario para utilizar el proyecto en BlueJ.**



B) En términos de extensiones de los archivos, se encuentra un archivo .astah, que contiene los diagramas del proyecto y adentro del directorio del punto anterior podemos encontrar: .Java, .BlueJ, .class y. CTXT adicional a un README.md.

2. Exploren el proyecto en BlueJ (a) ¿Cuántas clases tiene? (b) ¿Cuál es la relación entre ellas? (c) ¿Cuál es la clase principal de la aplicación? (d) ¿Cómo la reconocen? (e) ¿Cuál es la clase “diferente”? (f) ¿Cuál es su propósito?

a) Tiene 3 clases incluyendo la clase de prueba

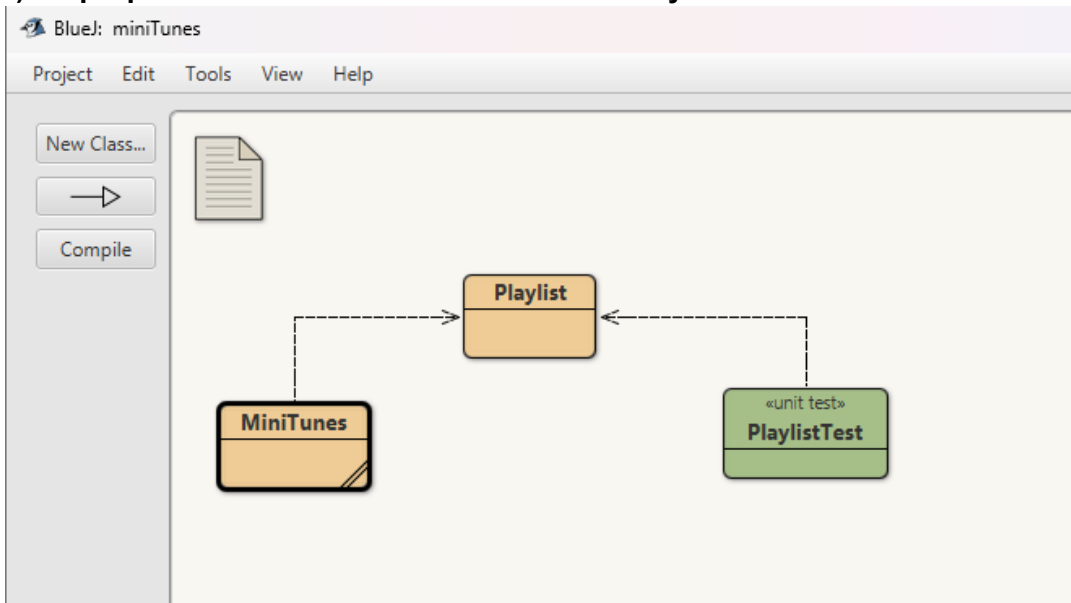
b) MiniTunes referencia a Playlist y PlaylistTest referencia a Playlist

c) Playlist

d) Porque las demás referencia a playlist, sin esta clase no habrian relaciones y quedarian separadas entre ellas

e) PlaylistTest es la clase diferente

f) Su propósito es realizar testeos a la clase Playlist



Para las siguientes dos preguntas sólo consideren las clases “normales”:

3. Generen y revisen la documentación del proyecto: ¿está completa la documentación de cada clase? (Detallen el estado de documentación: encabezado y métodos)

No está completa, falta la descripción de lo que hace cada método

Method Summary		
All Methods Instance Methods Concrete Methods		
Modifier and Type	Method	Description
void	<code>assign(String² a, ²String²[] playlist)</code>	
void	<code>assignBinary(String² a, ²String² b, ²char op, ²String² c)</code>	
void	<code>assignUnary(String² a, ²String² b, ²char op, ²String²[] values)</code>	
void	<code>define(String² name)</code>	
boolean	<code>ok()</code>	
int	<code>size(String² a)</code>	
String ²	<code>toString()</code>	
String ²	<code>toString(String² name)</code>	
Methods inherited from class <code>java.lang.Object</code>		
<code>clone², equals², getClass², hashCode², notify², notifyAll², wait², wait², wait²</code>		

4. Revisen el código fuente del proyecto, ¿en qué estado está cada clase? (Detallen el estado de las fuentes considerando dos dimensiones: (a) la primera, atributos y métodos, y la segunda, (b) código, documentación y comentarios) (c) ¿Qué diferencia hay entre el código, la documentación y los comentarios?
 - a) El código se presenta con la firma de los métodos, sin embargo, están sin implementación, adicional a eso, no se encuentran atributos declarados en las clases

```

import java.util.TreeMap;

/** MiniTunes.java
 *
 * @author ESCUELA 2026-02
 */

public class MiniTunes{

    private TreeMap<String,Playlist> playlists;

    public MiniTunes(){

    }

    //Define a new playlist name
    public void define(String name){

    }

    //Assign a playlist to an existing playlist name
    //a := playlist
    public void assign(String a, String [] [] playlist){

    }

    //Return a playlist's size
    public int size(String a){
        return -1;
    }

    //Returns the playlist names in alphabetical order. comma-separated
    public String toString(){
        return null;
    }

    // Returns the string representation of a playlist.
    public String toString(String name){
        return null;
    }
}

```

- b) Esta con varios comentarios definiendo que hace cada parte, también existen ciertas restricciones que son mencionadas dentro del código en (Playlist), pero la clase de PlaylistTest no tiene comentarios dentro de su código que explique funcionamiento, y en su documentación también está incompleta.

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
void	setUp()	Sets up the test fixture.
void	shouldCreateAEmptyPlaylist()	
void	shouldCreateAPlaylist()	
void	shouldNotCreateABadPlaylist()	
void	shouldRecognizeEqualPlaylists()	
void	tearDown()	Tears down the test fixture.

Methods inherited from class java.lang.Object

clone, equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

```
//Each song is described by its title, artist, genre, duration, and rating.
//The title and artist are mandatory. The genre, duration, and rating may be unknown.
//The combination (title, artist) must be unique. Two songs cannot have the same title and artist.
//The duration (minutes) must be between 1 and 9.
//The rating must be between * and *****.

public class Playlist {

    public Playlist(String [][] songs){
    }

    public Playlist add(String [] song){
        return null;
    }

    public Playlist delete(String [] song){
        return null;
    }

    public Playlist select(String [] values){
        return null;
    }

    public int size(){
        return -1;
    }

    // Songs are in uppercase with unnecessary spaces removed.
    // Columns are aligned and separated by three spaces.
    //TITLE  ARTIST      GENRE  DURATION  RATING
    //ONE    U2          ROCK   4         *****
    //NUMB   LINKIN PARK  ROCK   3         *****
    //ALIVE  PEARL JAM      ROCK   5         *****
    //CREEP  RADIOHEAD      ROCK   4         *****
    //DREAMS FLEETWOOD MAC    .       4         *****

    public String toString() {
```

- c) El código es diciente en sus firmas y encabezados, la documentación está incompleta puesto que solo aparece el código y su firma, en los comentarios es donde está ubicado toda la parte de la explicación de lo que hace el código.

B Realizando ingeniería inversa [En [lab02.pdf](#) [miniTunes.astah](#)]

MDD: MODEL DRIVEN DEVELOPMENT

1. Completen el diagrama de clases correspondiente al proyecto. (No incluyan la clase de pruebas) (FALTAN ATRIBUTOS, INCLUIR CAPTURAS)
2. (a) ¿Cuáles contenedores están definidos? (b) ¿Qué diferencias hay entre el nuevo contenedor y el ArrayList y el vector [] que conocemos? Consulte el API de java.

A) El único contenedor definido es el TreeMap



B) Diferencias y similitudes entre TreeMap, ArrayList y vector.

- **Tamaño:** TreeMap y ArrayList son dinámicos, mientras que el vector es de tamaño fijo
 - **Acceso:** TreeMap utiliza como método de acceso una clave, mientras que ArrayList y vector se acceden mediante el índice.
 - **Qué guarda:** TreeMap guarda pares clave-valor, mientras que ArrayList y vector guardan solo índices.
3. En el nuevo contenedor, (a) ¿Cómo adicionamos un elemento? (b) ¿Cómo lo consultamos? (c) ¿Cómo lo eliminamos?
 - A) Para agregar se usa el método `.put(clave,valor)`
 - B) Para consultar se usa el método `.get(clave)` y retorna el valor de la clave
 - C) Para eliminar se usa el método `.remove(clave)`

PARTE II. Aprendiendo a probar

A Conociendo Pruebas en BlueJ [En [lab02.pdf](#) [*.java](#)]

BDD

Para poder cumplir con la prácticas XP vamos a aprender a realizar las pruebas de unidad usando las herramientas apropiadas. Para eso implementaremos algunos métodos en la clase [Playlist](#).

1. Revisen el código de la clase [PlaylistTest](#) (a) ¿cuáles etiquetas tiene (componentes con símbolo @)? (b) ¿cuántos métodos tiene? (c) ¿cuántos métodos son de prueba? (d) ¿cómo los reconocen?
 - A) Contiene 6 etiquetas (4 etiquetas Test), (1 etiqueta before), (1 etiqueta after)
 - B) Tiene 6 métodos.
2. Ejecuten los tests de la clase [PlaylistTest](#). (click derecho sobre la clase, Test All) (a) ¿cuántas pruebas se ejecutan? (b) ¿cuántas pasan? (c) ¿por qué? (d) Capturen la pantalla.
3. Estudien las etiquetas encontradas en 1. Expliquen en sus palabras su significado.
4. Estudie los métodos [assertTrue](#), [assertFalse](#), [assertEquals](#), [assertNull](#) y [fail](#) de la clase [Assert](#) del API JUnit 1. Explique en sus palabras que hace cada uno de ellos.
5. Investiguen y expliquen la diferencia que entre un fallo y un error en [JUnit](#). Escriba código, usando los métodos del punto 4., para codificar los siguientes tres casos de prueba y lograr que se comporten como lo prometen [shouldPass](#), [shouldFail](#), [shouldErr](#).

B Practicando Pruebas en BlueJ [En [lab02.pdf](#) [*.java](#)]

BDD

Ahora vamos escribir el código necesario para que las pruebas de pasen [PlaylistTest](#).

1. (a) Determinen los atributos de la clase [Playlist](#). (b) Justifiquen la selección.
2. (a) Determinen el invariante de la clase [Playlist](#). (b) Justifiquen la decisión.
3. Implementen los métodos de [Playlist](#) necesarios para pasar todas las pruebas definidas. (a) ¿Cuáles métodos implementaron?
4. Capturen los resultados de las pruebas de unidad.

PARTE III. Desarrollando el proyecto `miniTunes`. [En `lab02.pdf` `miniTunes.astah *.java`]

MDD-BDD

Para desarrollar esta aplicación vamos a considerar algunos ciclos.

En cada ciclo deben realizar los pasos definidos a continuación.

- I Definir los métodos base de correspondientes al mini-ciclo actual.
- II Definir y programar los casos de prueba de esos métodos.
Piensen en los debería y los noDebería (`should...` y `shouldNot...`)
- III Diseñar los métodos
Usen diagramas de secuencia. En astah, creen el diagrama sobre el método correspondiente.
- IV Escribir el código correspondiente (no olvide la documentación)
- V Ejecutar las pruebas de unidad (vuelva a 3 (a veces a 2), si no están en verde)
- VI Completar la tabla de clases y métodos. (Al final del documento)

Ciclo 1 : Operaciones básicas: definir el nombre de una lista, asignar a un nombre una lista, consultar nombres de listas y consultar las canciones de una lista.

Ciclo 2: Operaciones unarias: adicionar una canción, eliminar una canción y seleccionar canciones dada una condición.

Ciclo 3 : Operaciones binarias: unión, intersección y diferencia.

BONO Ciclo 4 : Defina e implemente tres nuevas operaciones

Completen la siguiente tabla indicando el número de ciclo y los métodos asociados de cada clase.

Ciclo	MiniTunes	MiniTunesTest
-------	-----------	---------------

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.